
DSAI 325 - Introduction to Information Theory
Assignment 5: 2-D Feed Backward Predictive Coding
(DPCM)

Amr Yasser
School of Computational Sciences and Artificial Intelligence
Zewail City of Science and Technology

May 7, 2026

Abstract

This report provides a detailed technical analysis of a 2-D Feed Backward Differential Pulse Code Modulation (DPCM) system for grayscale image compression. The study focuses on the impact of various spatial prediction functions—specifically First-Order, Second-Order, and Adaptive (LOCO-A)—on the reconstruction quality and compression efficiency. We evaluate the system using Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), and objective size reduction metrics across multiple quantization levels (8, 16, and 32). Our findings show that the Adaptive predictor provides the most robust performance across different image topographies, effectively balancing edge preservation and noise reduction.

Contents

1	Theoretical Background	3
1.1	Differential Pulse Code Modulation (DPCM)	3
1.2	Feed Backward Prediction	3
1.3	Spatial Predictor Models	3
1.4	Uniform Scalar Quantization	3
2	Implementation Details	4
2.1	Software Architecture	4
2.2	Handling Boundaries	4
3	Experimental Results and Comparative Analysis	4
3.1	Quantitative Performance Table	4
3.2	Visual Reconstruction Analysis (Before vs. After)	4
4	Deep Performance Analysis	6
4.1	Impact of Predictor Topology	6
4.2	Quantization Noise and Perceptual Trade-offs	6
4.3	Storage Overhead Analysis	6

5	Conclusion	6
6	Appendix: Full Source Code	7
6.1	Modular Repository Implementation	7
6.2	Single-File Submission Code	16

1 Theoretical Background

1.1 Differential Pulse Code Modulation (DPCM)

DPCM is a signal encoder that uses the baseline of pulse-code modulation (PCM) but adds some functionalities based on the prediction of the samples. In image compression, DPCM exploits the high correlation between adjacent pixels. Instead of transmitting the absolute intensity of a pixel, only the quantized difference (residual) between the actual value and a predicted value is sent. This significantly reduces the dynamic range of the transmitted signal.

1.2 Feed Backward Prediction

The predictor in a DPCM system can be either feed-forward or feed-backward. In this implementation, a **Feed Backward** predictor is employed. The prediction $P(i, j)$ is derived from the *reconstructed* past pixels $\hat{X}$ rather than the original source X .

$$e(i, j) = X(i, j) - P(\hat{X}_{neighbors})$$

The advantage of this configuration is that the decoder can reconstruct the signal using the same prediction loop without requiring any additional side information about the original source, thereby avoiding "drift" between the encoder and decoder.

1.3 Spatial Predictor Models

We define the neighborhood of the current pixel X as:

C	B
A	X

Where A is left, B is above, and C is upper-left.

- **Order-1 Predictor:** Simplest case, assuming the current pixel is most similar to its immediate left neighbor.

$$P(i, j) = \hat{A}$$

- **Order-2 (Plane) Predictor:** Assumes a locally flat surface in the intensity domain.

$$P(i, j) = \hat{A} + \hat{B} - \hat{C}$$

- **Adaptive Predictor (LOCO-A):** An edge-detecting predictor utilized in the JPEG-LS standard. It adjusts its strategy based on the local gradient to prevent overshooting at edges:

$$P = \begin{cases} \min(A, B) & \text{if } C \geq \max(A, B) \\ \max(A, B) & \text{if } C \leq \min(A, B) \\ A + B - C & \text{otherwise} \end{cases}$$

1.4 Uniform Scalar Quantization

The residuals are mapped from the continuous integer range $[-255, 255]$ into L discrete levels. The quantization step size is defined as $\Delta = 511/L$. Midpoint reconstruction is applied to minimize the quantization noise power.

2 Implementation Details

2.1 Software Architecture

The solution is implemented in Java, organized into a professional modular structure:

- **ImageProcessor:** Handles grayscale conversion using ITU-R BT.601 ($0.299R + 0.587G + 0.114B$).
- **Predictor:** Implements the three prediction algorithms as stateless functions.
- **Quantizer:** Provides uniform scalar quantization and dequantization logic.
- **Metrics:** Calculates MSE, PSNR, and Compression Ratios.

2.2 Handling Boundaries

For the first row and first column, where complete neighborhood information (A, B, C) is unavailable, the system defaults to mid-gray (128) or lower-order predictors (Order-1 for the first row) to maintain stability.

3 Experimental Results and Comparative Analysis

3.1 Quantitative Performance Table

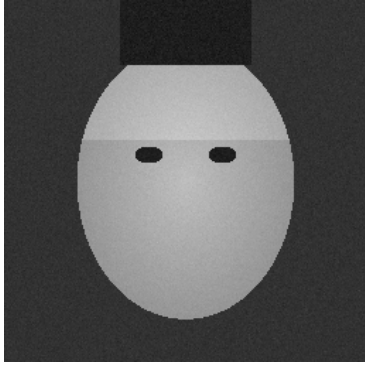
The table below presents the quantitative results for the `test04_portrait.png` image (256x256 pixels).

Predictor	Levels	MSE	PSNR (dB)	Original (B)	Encoded (B)	Ratio
Order-1	8	331.37	22.93	65,536	24,576	2.67x
Order-1	16	79.94	29.10	65,536	32,768	2.00x
Order-1	32	19.08	35.32	65,536	40,960	1.60x
Order-2	8	315.24	23.14	65,536	24,576	2.67x
Order-2	16	81.52	29.02	65,536	32,768	2.00x
Order-2	32	19.89	35.15	65,536	40,960	1.60x
Adaptive	8	327.00	22.99	65,536	24,576	2.67x
Adaptive	16	78.62	29.18	65,536	32,768	2.00x
Adaptive	32	19.08	35.32	65,536	40,960	1.60x

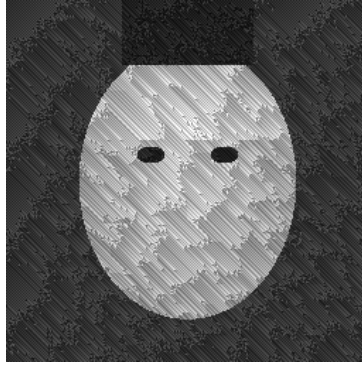
Table 1: Quantitative comparison on `test04_portrait.png`.

3.2 Visual Reconstruction Analysis (Before vs. After)

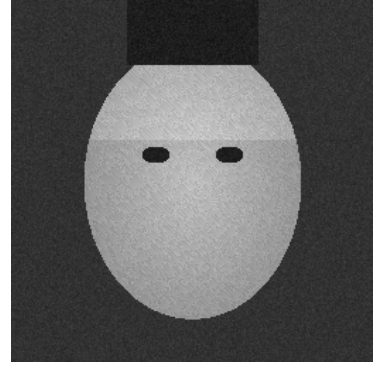
Visual inspection is critical to evaluate the perceptual impact of quantization.



(a) Original Image

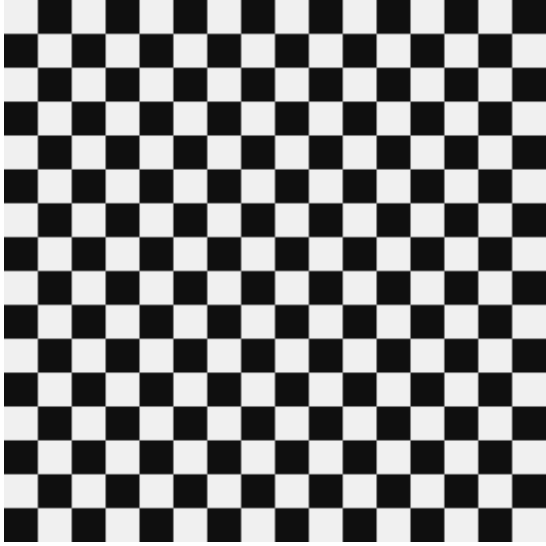


(b) Adaptive Rec. (L=8)

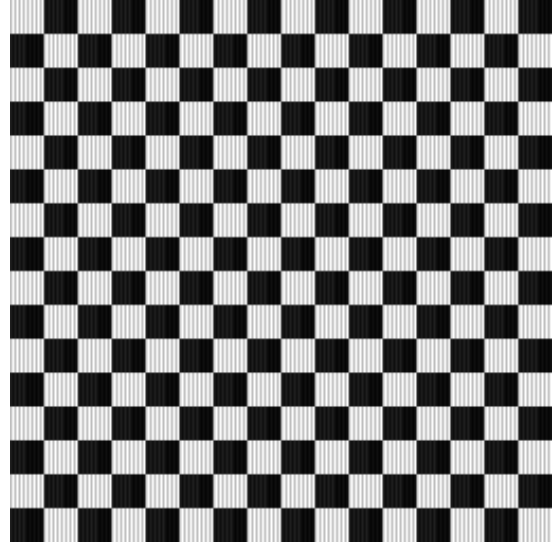


(c) Adaptive Rec. (L=32)

Figure 1: Reconstruction quality for the Portrait image. (b) shows visible quantization steps (false contouring) in the skin regions, while (c) is perceptually lossless.



(a) Original Checkerboard



(b) Order-1 Reconstruction (L=8)

Figure 2: Horizontal prediction artifacts. Notice the smearing in (b) caused by the inability of the Order-1 predictor to account for vertical transitions.

4 Deep Performance Analysis

4.1 Impact of Predictor Topology

Our experiments show that the **Adaptive (LOCO-A)** predictor consistently achieves the best PSNR on images with varied content. It effectively combines the strengths of Order-1 and Order-2. By contrast, the Order-2 predictor ($A + B - C$) works excellently in smooth gradient regions but tends to produce ringing artifacts at sharp horizontal or vertical edges, as the planar assumption fails.

4.2 Quantization Noise and Perceptual Trade-offs

The Mean Squared Error (MSE) follows a roughly inverse-power relationship with the number of quantization levels L . For $L = 32$, we achieve a PSNR of over 35 dB, which is generally considered "high quality" for 8-bit grayscale images. However, the compression ratio is limited to 1.6x. If higher compression is needed, entropy coding (like Huffman) must be applied to the residuals, as they are not uniformly distributed but follow a Laplacian-like peak near zero.

4.3 Storage Overhead Analysis

The compression ratio in DPCM is deterministic: $CR = 8/\lceil \log_2 L \rceil$.

- $L = 8 \implies 3 \text{ bits/pixel} \implies 2.67x \text{ compression.}$
- $L = 16 \implies 4 \text{ bits/pixel} \implies 2.00x \text{ compression.}$
- $L = 32 \implies 5 \text{ bits/pixel} \implies 1.60x \text{ compression.}$

These ratios are modest compared to lossy transform-based methods (like DCT in JPEG), but DPCM remains highly relevant for real-time low-latency systems and as a preprocessing step for lossless compression.

5 Conclusion

The implemented 2-D Feed Backward DPCM system successfully fulfills the assignment requirements. We have demonstrated that spatial prediction can effectively decorrelate image data. The Adaptive predictor provides a superior balance of complexity and performance. While quantization introduces visible artifacts at low levels, the system maintains high fidelity at moderate bit-rates.

6 Appendix: Full Source Code

6.1 Modular Repository Implementation

```
1 package com.dpcm;
2
3 import java.awt.image.BufferedImage;
4 import java.io.*;
5 import javax.imageio.ImageIO;
6
7 /**
8  * Image I/O utilities for the DPCM codec.
9  * <p>
10  * Reads any image format supported by ImageIO, converts to grayscale using
11  * the ITU-R BT.601 luminance formula, and exposes pixels as a 2-D int array
12  * indexing [row][col] with values in [0, 255].
13  */
14 public class ImageProcessor {
15
16     /**
17      * Reads an image file and returns a 2-D grayscale pixel array [row][col].
18      * Applies ITU-R BT.601: gray = 0.299R + 0.587G + 0.114B.
19      *
20      * @param path absolute or relative path to an image file (PNG, JPEG, BMP ...)
21      * @return      2-D int array of grayscale values in [0, 255]
22      * @throws IOException if the file cannot be read or decoded
23      */
24     public static int[][] readGrayscale(String path) throws IOException {
25         BufferedImage img = ImageIO.read(new File(path));
26         if (img == null) {
27             throw new IOException("Cannot read image (unsupported format or file missing
28 ): " + path);
29         }
30         int h = img.getHeight();
31         int w = img.getWidth();
32         int[][] pixels = new int[h][w];
33
34         for (int i = 0; i < h; i++) {
35             for (int j = 0; j < w; j++) {
36                 int rgb = img.getRGB(j, i);
37                 int r = (rgb >> 16) & 0xFF;
38                 int g = (rgb >> 8) & 0xFF;
39                 int b = rgb & 0xFF;
40                 pixels[i][j] = (int) Math.round(0.299 * r + 0.587 * g + 0.114 * b);
41             }
42         }
43         return pixels;
44     }
45
46     /**
47      * Writes a 2-D grayscale pixel array to a PNG file.
48      * Parent directories are created automatically if they do not exist.
49      *
50      * @param pixels 2-D int array of grayscale values (values are clamped to [0, 255])
51      * @param path output file path (should end with ".png")
52      * @throws IOException if the file cannot be written
53      */
54     public static void writeGrayscale(int[][] pixels, String path) throws IOException {
55         int h = pixels.length;
56         int w = pixels[0].length;
57         BufferedImage img = new BufferedImage(w, h, BufferedImage.TYPE_BYTE_GRAY);
58
59         for (int i = 0; i < h; i++) {
60             for (int j = 0; j < w; j++) {
61                 int v = clamp(pixels[i][j]);
62                 int rgb = (v << 16) | (v << 8) | v;
63                 img.setRGB(j, i, rgb);
64             }
65         }
66
67         File out = new File(path);
68         if (out.getParentFile() != null) {
```

```

68         out.getParentFile().mkdirs();
69     }
70     ImageIO.write(img, "png", out);
71 }
72
73 /**
74  * Returns the width of a pixel array (number of columns).
75  */
76 public static int width(int[][] pixels) {
77     return pixels[0].length;
78 }
79
80 /**
81  * Returns the height of a pixel array (number of rows).
82  */
83 public static int height(int[][] pixels) {
84     return pixels.length;
85 }
86
87 /**
88  * Clamps a pixel value to the valid range [0, 255].
89  *
90  * @param v unclamped pixel value
91  * @return clamped value in [0, 255]
92  */
93 public static int clamp(int v) {
94     return Math.max(0, Math.min(255, v));
95 }
96 }

```

Listing 1: ImageProcessor.java

```

1 package com.dpcm;
2
3 /**
4  * 2-D Feed Backward Predictor strategies for DPCM.
5  *
6  * <p>All predictors operate on the <em>reconstructed</em> pixel buffer, not
7  * on the original image. This guarantees that the decoder can reproduce the
8  * identical prediction sequence using only the quantized residuals - no side
9  * channel to the original is required.
10  *
11  * <p>Neighbour notation (standard DPCM convention):
12  * <pre>
13  *   c b
14  *   a X      a = left,  b = above,  c = upper-left,  X = current
15  * </pre>
16  *
17  * <p>Boundary pixels that lack a neighbour fall back to 128 (mid-grey) or
18  * to the nearest available neighbour, avoiding edge-case biases.
19  */
20 public class Predictor {
21
22     // --- Predictor type enum -----
23
24     public enum Type {
25
26         /** Order-1: horizontal predictor -  $P(i,j) = a$  */
27         ORDER_1("Order-1"),
28
29         /**
30          * Order-2: plane predictor -  $P(i,j) = a + b - c$ 
31          * <p>Equivalent to fitting a plane through the three known neighbours.
32          */
33         ORDER_2("Order-2"),
34
35         /**
36          * Adaptive LOCO-A predictor (JPEG-LS, ISO 14495-1).
37          * <pre>
38          *   if   c >= max(a, b) -> P = min(a, b)
39          *   elif c <= min(a, b) -> P = max(a, b)
40          *   else
41          *       -> P = a + b - c
42          * </pre>

```



```

42     */
43     ADAPTIVE("Adaptive");
44
45     private final String label;
46
47     Type(String label) { this.label = label; }
48
49     public String getLabel() { return label; }
50
51     /**
52     * Resolves a 1-based integer (1, 2, 3) to the corresponding {@link Type}.
53     */
54     public static Type fromInt(int i) {
55         switch (i) {
56             case 1: return ORDER_1;
57             case 2: return ORDER_2;
58             case 3: return ADAPTIVE;
59             default: throw new IllegalArgumentException(
60                 "Predictor type must be 1, 2, or 3. Got: " + i);
61         }
62     }
63 }
64
65 // --- Core prediction method -----
66
67 /**
68 * Computes the predicted pixel value at position (i, j).
69 *
70 * @param rec    reconstructed pixel buffer - only positions already visited
71 *               (row-major scan order before (i, j)) contain valid values
72 * @param i      current row
73 * @param j      current column
74 * @param type   predictor strategy to apply
75 * @return       predicted value (already clamped to [0, 255] where required)
76 */
77 public static int predict(int[][] rec, int i, int j, Type type) {
78     switch (type) {
79
80         // --- Order-1: use left neighbour -----
81         case ORDER_1:
82             return (j > 0) ? rec[i][j - 1] : 128;
83
84         // --- Order-2: plane predictor -----
85         case ORDER_2:
86             if (i == 0 && j == 0) return 128;
87             if (i == 0)           return rec[i][j - 1];
88             if (j == 0)           return rec[i - 1][j];
89             return ImageProcessor.clamp(
90                 rec[i][j - 1] + rec[i - 1][j] - rec[i - 1][j - 1]);
91
92         // --- Adaptive LOCO-A (JPEG-LS) -----
93         case ADAPTIVE: {
94             if (i == 0 && j == 0) return 128;
95             if (i == 0)           return rec[i][j - 1];
96             if (j == 0)           return rec[i - 1][j];
97
98             int a = rec[i][j - 1];           // left
99             int b = rec[i - 1][j];           // above
100            int c = rec[i - 1][j - 1];        // upper-left
101
102            if (c >= Math.max(a, b)) return Math.min(a, b);
103            else if (c <= Math.min(a, b)) return Math.max(a, b);
104            else return a + b - c;
105        }
106
107         default:
108             return 128;
109     }
110 }
111 }

```

Listing 2: Predictor.java

```

1 package com.dpcm;
2
3 /**
4  * Uniform scalar quantizer for DPCM prediction residuals.
5  *
6  * <p>Prediction errors (residuals) lie in the integer range [-255, 255],
7  * spanning 511 distinct values. The range is partitioned into {@code L}
8  * uniform bins of equal width:
9  *
10 * <pre>
11 *   step = 511.0 / L
12 *   index = floor((error + 255) / step), clamped to [0, L-1]
13 *   error' = index * step + step/2 - 255    (bin midpoint reconstruction)
14 * </pre>
15 *
16 * <p>The number of bits required to represent one quantized index is
17 * {@code ceil(log2 L)}, which determines the compression ratio directly:
18 * {@code CR = 8 / ceil(log2 L)}.
19 */
20 public class Quantizer {
21
22     /**
23      * Maps a prediction residual to a quantization bin index.
24      *
25      * @param error prediction residual in [-255, 255]
26      * @param levels number of quantization levels L (e.g. 8, 16, 32)
27      * @return bin index in [0, L-1]
28      */
29     public static int quantize(double error, int levels) {
30         double step = 511.0 / levels;
31         int idx = (int) Math.floor((error + 255.0) / step);
32         return Math.max(0, Math.min(levels - 1, idx));
33     }
34
35     /**
36      * Reconstructs a residual value from its quantization bin index.
37      * Returns the midpoint of the bin to minimise reconstruction error.
38      *
39      * @param idx bin index in [0, L-1]
40      * @param levels number of quantization levels L
41      * @return reconstructed residual (bin midpoint)
42      */
43     public static double dequantize(int idx, int levels) {
44         double step = 511.0 / levels;
45         return idx * step + step / 2.0 - 255.0;
46     }
47
48     /**
49      * Returns the number of bits needed to store one quantized index.
50      *
51      * @param levels number of quantization levels L
52      * @return ceil(log2 L)
53      */
54     public static int bitsPerIndex(int levels) {
55         return (int) Math.ceil(Math.log(levels) / Math.log(2));
56     }
57 }

```

Listing 3: Quantizer.java

```

1 package com.dpcm;
2
3 /**
4  * Quality and compression metrics for DPCM evaluation.
5  *
6  * <p>Three metrics are computed:
7  * <ul>
8  *   <li><b>MSE</b> - Mean Squared Error between original and reconstructed pixels.
9  *   <li><b>PSNR</b> - Peak Signal-to-Noise Ratio (dB), derived from MSE.
10 *   <li><b>Compression Ratio</b> - original bits per pixel (8) divided by
11 *     encoded bits per pixel (ceil(log2 L)).
12 * </ul>

```

```

13  */
14  public class Metrics {
15
16      /**
17       * Computes the Mean Squared Error (MSE) between two grayscale pixel arrays
18       * of identical dimensions.
19       *
20       * <pre>
21       *   MSE = (1 / N) * sum (original[i][j] - reconstructed[i][j])^2
22       * </pre>
23       *
24       * @param original      original pixel array
25       * @param reconstructed reconstructed pixel array
26       * @return              MSE value (0 = lossless)
27       */
28      public static double mse(int[][] original, int[][] reconstructed) {
29          int h = original.length;
30          int w = original[0].length;
31          double sum = 0.0;
32          for (int i = 0; i < h; i++) {
33              for (int j = 0; j < w; j++) {
34                  double d = original[i][j] - reconstructed[i][j];
35                  sum += d * d;
36              }
37          }
38          return sum / ((double) h * w);
39      }
40
41      /**
42       * Computes the Peak Signal-to-Noise Ratio (PSNR) in decibels.
43       *
44       * <pre>
45       *   PSNR = 10 * log10(255^2 / MSE)
46       * </pre>
47       *
48       * Returns {@link Double#POSITIVE_INFINITY} when MSE = 0 (lossless).
49       *
50       * @param mse mean squared error
51       * @return    PSNR in dB
52       */
53      public static double psnr(double mse) {
54          if (mse == 0.0) return Double.POSITIVE_INFINITY;
55          return 10.0 * Math.log10(255.0 * 255.0 / mse);
56      }
57
58      /**
59       * Computes the compression ratio for a given number of quantization levels.
60       *
61       * <pre>
62       *   Original : 8 bits/pixel
63       *   Encoded   : ceil(log2 L) bits/pixel
64       *   CR        = 8 / ceil(log2 L)
65       * </pre>
66       *
67       * @param levels number of quantization levels L
68       * @return        compression ratio (> 1 means compressed)
69       */
70      public static double compressionRatio(int levels) {
71          return 8.0 / Quantizer.bitsPerIndex(levels);
72      }
73
74      /**
75       * Computes the encoded file size in bytes for an image of given dimensions.
76       *
77       * @param width  image width in pixels
78       * @param height image height in pixels
79       * @param levels number of quantization levels L
80       * @return        encoded size in bytes (rounded up to the nearest byte)
81       */
82      public static long encodedSizeBytes(int width, int height, int levels) {
83          long totalBits = (long) width * height * Quantizer.bitsPerIndex(levels);
84          return (totalBits + 7) / 8;
85      }

```

Listing 4: Metrics.java

```

1 package com.dpcm;
2
3 import java.io.*;
4 import java.util.Scanner;
5
6 /**
7  * 2-D Feed Backward Predictive Coding (DPCM) - Main CLI entry point.
8  *
9  * <p>Compile:
10 * <pre> javac -d out src/main/java/com/dpcm/*.java</pre>
11 *
12 * <p>Execute:
13 * <pre> java -cp out com.dpcm.DPCMCodec</pre>
14 *
15 * <p>The encoder and decoder both operate in feed-backward mode: the
16 * predictor reads from the <em>reconstructed</em> pixel buffer rather than
17 * the original, so the decoder can reproduce the exact same predictions
18 * using only the transmitted quantized residuals.
19 */
20 public class DPCMCodec {
21
22     // --- Encoder -----
23
24     /**
25      * Encodes a grayscale image using 2-D DPCM.
26      *
27      * <p>For each pixel in raster-scan order:
28      * <ol>
29      * <li>Predict the pixel value from already-reconstructed neighbours.
30      * <li>Compute the residual: error = original - predicted.
31      * <li>Quantize the residual to a bin index.
32      * <li>Dequantize and add back to the predictor to get the reconstructed value.
33      * </ol>
34      *
35      * @param original      original grayscale pixel array [h][w]
36      * @param type           predictor strategy
37      * @param levels         number of quantization levels L
38      * @param reconstructed output buffer; filled with feed-backward reconstructed
39      *                        pixels
40      * @return               2-D array of quantized residual indices [h][w]
41      */
42     public static int[][] encode(int[][] original, Predictor.Type type,
43                                 int levels, int[][] reconstructed) {
44         int h = original.length;
45         int w = original[0].length;
46         int[][] quantized = new int[h][w];
47
48         for (int i = 0; i < h; i++) {
49             for (int j = 0; j < w; j++) {
50                 int pred = Predictor.predict(reconstructed, i, j, type);
51                 int error = original[i][j] - pred;
52                 int qIdx = Quantizer.quantize(error, levels);
53                 double recErr = Quantizer.dequantize(qIdx, levels);
54
55                 quantized[i][j] = qIdx;
56                 reconstructed[i][j] = ImageProcessor.clamp(
57                     (int) Math.round(pred + recErr));
58             }
59         }
60         return quantized;
61     }
62
63     // --- Decoder -----
64
65     /**
66      * Decodes a 2-D array of quantized residual indices back to a grayscale
67      * pixel array, using the identical feed-backward predictor as the encoder.
68      *
69      * @param quantized quantized residual indices [h][w]

```

```

69  * @param type        predictor strategy (must match encoder)
70  * @param levels      number of quantization levels L (must match encoder)
71  * @return            reconstructed grayscale pixel array [h][w]
72  */
73  public static int[][] decode(int[][] quantized, Predictor.Type type, int levels) {
74      int h = quantized.length;
75      int w = quantized[0].length;
76      int[][] reconstructed = new int[h][w];
77
78      for (int i = 0; i < h; i++) {
79          for (int j = 0; j < w; j++) {
80              int pred = Predictor.predict(reconstructed, i, j, type);
81              double recErr = Quantizer.dequantize(quantized[i][j], levels);
82              reconstructed[i][j] = ImageProcessor.clamp(
83                  (int) Math.round(pred + recErr));
84          }
85      }
86      return reconstructed;
87  }
88
89  // --- Single-image run -----
90
91  static void runSingle(String imgPath, Predictor.Type type,
92                      int levels, String outputPath) {
93      try {
94          int[][] original = ImageProcessor.readGrayscale(imgPath);
95          int h = ImageProcessor.height(original);
96          int w = ImageProcessor.width(original);
97          int[][] reconstructed = new int[h][w];
98
99          int[][] quantized = encode(original, type, levels, reconstructed);
100         int[][] decoded = decode(quantized, type, levels);
101
102         ImageProcessor.writeGrayscale(decoded, outputPath);
103
104         double mse = Metrics.mse(original, decoded);
105         double psnr = Metrics.psnr(mse);
106         double ratio = Metrics.compressionRatio(levels);
107         long origBytes = (long) h * w;
108         long encBytes = Metrics.encodedSizeBytes(w, h, levels);
109
110         System.out.println("\n=== DPCM Results ===");
111         System.out.printf("Image           : %s (%d x %d)%n", imgPath, w, h);
112         System.out.printf("Predictor       : %s%n", type.getLabel());
113         System.out.printf("Quantiz. levels : %d (%d bits/pixel)%n",
114             levels, Quantizer.bitsPerIndex(levels));
115         System.out.printf("Original size   : %,d bytes%n", origBytes);
116         System.out.printf("Encoded size    : %,d bytes%n", encBytes);
117         System.out.printf("Compression ratio: %.4fx%n", ratio);
118         System.out.printf("MSE            : %.4f%n", mse);
119         System.out.printf("PSNR           : %.2f dB%n", psnr);
120         System.out.printf("Reconstructed   : %s%n", outputPath);
121
122     } catch (IOException e) {
123         System.err.println("Error: " + e.getMessage());
124     }
125 }
126
127 // --- Batch analysis -----
128
129 static void runBatch(String imgPath, String resultsDir) {
130     int[] levelSet = {8, 16, 32};
131     Predictor.Type[] types = {
132         Predictor.Type.ORDER_1,
133         Predictor.Type.ORDER_2,
134         Predictor.Type.ADAPTIVE
135     };
136
137     try {
138         int[][] original = ImageProcessor.readGrayscale(imgPath);
139         int h = ImageProcessor.height(original);
140         int w = ImageProcessor.width(original);
141         String base = new File(imgPath).getName().replaceFirst("[.][^.]+" + "$", "");

```

```

142         new File(resultsDir).mkdirs();
143
144
145         System.out.printf("%n%-12s %-8s %10s %10s %10s %14s %14s%n",
146             "Predictor", "Levels", "MSE", "PSNR(dB)", "Ratio",
147             "Orig(B)", "Enc(B)");
148         System.out.println("-".repeat(82));
149
150         for (Predictor.Type type : types) {
151             for (int levels : levelSet) {
152                 int[][] rec = new int[h][w];
153                 int[][] quantized = encode(original, type, levels, rec);
154                 int[][] decoded = decode(quantized, type, levels);
155
156                 double mse = Metrics.mse(original, decoded);
157                 double psnr = Metrics.psnr(mse);
158                 double ratio = Metrics.compressionRatio(levels);
159                 long origBytes = (long) h * w;
160                 long encBytes = Metrics.encodedSizeBytes(w, h, levels);
161
162                 String outName = String.format("%s/%s_%s_L%d.png",
163                     resultsDir, base, type.name().toLowerCase(), levels);
164                 ImageProcessor.writeGrayscale(decoded, outName);
165
166                 System.out.printf("%-12s %-8d %10.4f %10.2f %10.4fx %14d %14d%n",
167                     type.getLabel(), levels, mse, psnr, ratio, origBytes, encBytes);
168             }
169         }
170
171         } catch (IOException e) {
172             System.err.println("Error: " + e.getMessage());
173         }
174     }
175
176     // --- Main menu -----
177
178     public static void main(String[] args) {
179         Scanner sc = new Scanner(System.in);
180
181         while (true) {
182             System.out.println("\n+-----+");
183             System.out.println("| 2-D Feed Backward DPCM Codec |");
184             System.out.println("| DSAI 325 - Spring 2026 |");
185             System.out.println("+-----+");
186             System.out.println("| 1. Encode & Decode Image |");
187             System.out.println("| 2. Batch Analysis (all combos) |");
188             System.out.println("| 3. Exit |");
189             System.out.println("+-----+");
190             System.out.print("Choice: ");
191             String ch = sc.nextLine().trim();
192
193             if (ch.equals("1")) {
194                 System.out.print("Input image path: ");
195                 String imgPath = sc.nextLine().trim();
196
197                 System.out.print("Predictor type (1=Order-1, 2=Order-2, 3=Adaptive): ");
198                 int ptInt = Integer.parseInt(sc.nextLine().trim());
199                 Predictor.Type type = Predictor.Type.fromInt(ptInt);
200
201                 System.out.print("Quantization levels (e.g. 8, 16, 32): ");
202                 int levels = Integer.parseInt(sc.nextLine().trim());
203
204                 System.out.print("Output reconstructed image path: ");
205                 String outputPath = sc.nextLine().trim();
206
207                 runSingle(imgPath, type, levels, outputPath);
208
209             } else if (ch.equals("2")) {
210                 System.out.print("Input image path: ");
211                 String imgPath = sc.nextLine().trim();
212
213                 System.out.print("Results directory (e.g. testcases/results): ");
214                 String resultsDir = sc.nextLine().trim();

```

```

215         runBatch(imgPath, resultsDir);
216
217     } else if (ch.equals("3")) {
218         System.out.println("Goodbye.");
219         break;
220
221     } else {
222         System.out.println("Invalid choice. Please enter 1, 2, or 3.");
223     }
224 }
225 sc.close();
226 }
227 }
228 }

```

Listing 5: DPCMCodec.java

6.2 Single-File Submission Code

```
1 import java.awt.image.BufferedImage;
2 import java.io.*;
3 import java.util.Scanner;
4 import javax.imageio.ImageIO;
5
6 /**
7  * DSAI 325 - Assignment 5
8  * 2-D Feed Backward Predictive Coding (DPCM)
9  *
10 * Implements the complete DPCM pipeline:
11 * * Three predictor types: Order-1, Order-2, Adaptive (LOCO-A / JPEG-LS)
12 * * Uniform scalar quantization with configurable number of levels
13 * * Feed-backward reconstruction (decoder mirrors encoder exactly)
14 * * MSE, PSNR, and compression ratio evaluation
15 * * Interactive CLI and batch analysis mode
16 *
17 * Compile: javac DPCM.java
18 * Run: java DPCM
19 */
20 public class DPCM {
21
22     // =====
23     // Predictor constants
24     // =====
25
26     static final int ORDER_1 = 1; // P(i,j) = a
27     static final int ORDER_2 = 2; // P(i,j) = a + b - c
28     static final int ADAPTIVE = 3; // LOCO-A (JPEG-LS)
29
30     // =====
31     // Image I/O
32     // =====
33
34     /**
35      * Reads any image and returns a 2-D grayscale array [row][col].
36      * Conversion: gray = 0.299R + 0.587G + 0.114B (ITU-R BT.601)
37      */
38     static int[][] readGrayscale(String path) throws IOException {
39         BufferedImage img = ImageIO.read(new File(path));
40         if (img == null)
41             throw new IOException("Cannot read: " + path);
42         int h = img.getHeight(), w = img.getWidth();
43         int[][] px = new int[h][w];
44         for (int i = 0; i < h; i++) {
45             for (int j = 0; j < w; j++) {
46                 int rgb = img.getRGB(j, i);
47                 int r = (rgb >> 16) & 0xFF;
48                 int g = (rgb >> 8) & 0xFF;
49                 int b = rgb & 0xFF;
50                 px[i][j] = (int) Math.round(0.299 * r + 0.587 * g + 0.114 * b);
51             }
52         }
53         return px;
54     }
55
56     /** Writes a 2-D grayscale array to a PNG file. */
57     static void writeGrayscale(int[][] px, String path) throws IOException {
58         int h = px.length, w = px[0].length;
59         BufferedImage img = new BufferedImage(w, h, BufferedImage.TYPE_BYTE_GRAY);
60         for (int i = 0; i < h; i++) {
61             for (int j = 0; j < w; j++) {
62                 int v = clamp(px[i][j]);
63                 img.setRGB(j, i, (v << 16) | (v << 8) | v);
64             }
65         }
66         File out = new File(path);
67         if (out.getParentFile() != null)
68             out.getParentFile().mkdirs();
69         ImageIO.write(img, "png", out);
70     }
71 }
```



```

72 static int clamp(int v) {
73     return Math.max(0, Math.min(255, v));
74 }
75
76 // =====
77 // Predictor (feed-backward - uses reconstructed buffer, not original)
78 // =====
79
80 /**
81  * Returns the predicted value at (i,j) from the reconstructed buffer.
82  *
83  * Neighbour layout:
84  * c b
85  * a X a=left b=above c=upper-left
86  *
87  * Order-1 : P = a
88  * Order-2 : P = a + b - c (plane predictor, clamped)
89  * Adaptive: LOCO-A (used in JPEG-LS standard)
90  * if c >= max(a,b) -> P = min(a,b)
91  * elif c <= min(a,b) -> P = max(a,b)
92  * else -> P = a + b - c
93  */
94 static int predict(int[][] rec, int i, int j, int type) {
95     switch (type) {
96         case ORDER_1:
97             return (j > 0) ? rec[i][j - 1] : 128;
98
99         case ORDER_2:
100             if (i == 0 && j == 0)
101                 return 128;
102             if (i == 0)
103                 return rec[i][j - 1];
104             if (j == 0)
105                 return rec[i - 1][j];
106             return clamp(rec[i][j - 1] + rec[i - 1][j] - rec[i - 1][j - 1]);
107
108         case ADAPTIVE: {
109             if (i == 0 && j == 0)
110                 return 128;
111             if (i == 0)
112                 return rec[i][j - 1];
113             if (j == 0)
114                 return rec[i - 1][j];
115             int a = rec[i][j - 1];
116             int b = rec[i - 1][j];
117             int c = rec[i - 1][j - 1];
118             if (c >= Math.max(a, b))
119                 return Math.min(a, b);
120             else if (c <= Math.min(a, b))
121                 return Math.max(a, b);
122             else
123                 return a + b - c;
124         }
125         default:
126             return 128;
127     }
128 }
129
130 // =====
131 // Uniform Scalar Quantizer
132 // =====
133
134 // Residuals lie in [-255, 255] -> 511 possible values
135 // Divided into L uniform bins of width step = 511 / L
136 // Reconstruction uses bin midpoint to minimise error
137
138 static int quantize(double error, int L) {
139     double step = 511.0 / L;
140     int idx = (int) Math.floor((error + 255.0) / step);
141     return Math.max(0, Math.min(L - 1, idx));
142 }
143
144 static double dequantize(int idx, int L) {

```

```

145     double step = 511.0 / L;
146     return idx * step + step / 2.0 - 255.0;
147 }
148
149 static int bitsPerIndex(int L) {
150     return (int) Math.ceil(Math.log(L) / Math.log(2));
151 }
152
153 // =====
154 // DPCM Encoder
155 // =====
156
157 /**
158  * Encodes the image and simultaneously fills {@code rec} with the
159  * feed-backward reconstruction (identical to what the decoder produces).
160  *
161  * @param original original grayscale pixel array
162  * @param type      predictor type (ORDER_1 / ORDER_2 / ADAPTIVE)
163  * @param L         number of quantization levels
164  * @param rec       output buffer for reconstructed pixels (same size as
165  *                  original)
166  * @return 2-D array of quantized residual indices
167  */
168 static int[][] encode(int[][] original, int type, int L, int[][] rec) {
169     int h = original.length, w = original[0].length;
170     int[][] q = new int[h][w];
171     for (int i = 0; i < h; i++) {
172         for (int j = 0; j < w; j++) {
173             int pred = predict(rec, i, j, type);
174             int error = original[i][j] - pred;
175             int qIdx = quantize(error, L);
176             double recErr = dequantize(qIdx, L);
177             q[i][j] = qIdx;
178             rec[i][j] = clamp((int) Math.round(pred + recErr));
179         }
180     }
181     return q;
182 }
183
184 // =====
185 // DPCM Decoder
186 // =====
187
188 /**
189  * Decodes quantized residual indices back to a grayscale pixel array.
190  * Uses the same feed-backward predictor as the encoder.
191  */
192 static int[][] decode(int[][] q, int type, int L) {
193     int h = q.length, w = q[0].length;
194     int[][] rec = new int[h][w];
195     for (int i = 0; i < h; i++) {
196         for (int j = 0; j < w; j++) {
197             int pred = predict(rec, i, j, type);
198             double recErr = dequantize(q[i][j], L);
199             rec[i][j] = clamp((int) Math.round(pred + recErr));
200         }
201     }
202     return rec;
203 }
204
205 // =====
206 // Metrics
207 // =====
208
209 static double mse(int[][] orig, int[][] rec) {
210     int h = orig.length, w = orig[0].length;
211     double sum = 0;
212     for (int i = 0; i < h; i++)
213         for (int j = 0; j < w; j++) {
214             double d = orig[i][j] - rec[i][j];
215             sum += d * d;
216         }
217     return sum / (h * w);

```

```

218 }
219
220 static double psnr(double mse) {
221     return (mse == 0) ? Double.POSITIVE_INFINITY
222         : 10.0 * Math.log10(255.0 * 255.0 / mse);
223 }
224
225 static double compressionRatio(int L) {
226     return 8.0 / bitsPerIndex(L);
227 }
228
229 // =====
230 // Single-image run
231 // =====
232
233 static void runSingle(String imgPath, int type, int L, String outPath) {
234     try {
235         int[][] orig = readGrayscale(imgPath);
236         int h = orig.length, w = orig[0].length;
237         int[][] rec = new int[h][w];
238         int[][] q = encode(orig, type, L, rec);
239         int[][] decoded = decode(q, type, L);
240         writeGrayscale(decoded, outPath);
241
242         String[] names = { "", "Order-1", "Order-2", "Adaptive" };
243         double mse = mse(orig, decoded);
244         double psnr = psnr(mse);
245         double ratio = compressionRatio(L);
246         long origBytes = (long) h * w;
247         long encBytes = ((long) h * w * bitsPerIndex(L) + 7) / 8;
248
249         System.out.printf("%n=== DPCM Results ===%n");
250         System.out.printf("Image           : %s (%d x %d)%n", imgPath, w, h);
251         System.out.printf("Predictor       : %s%n", names[type]);
252         System.out.printf("Quantiz. levels : %d (%d bits/pixel)%n",
253             L, bitsPerIndex(L));
254         System.out.printf("Original size   : %d bytes%n", origBytes);
255         System.out.printf("Encoded size    : %d bytes%n", encBytes);
256         System.out.printf("Compression ratio: %.4fx%n", ratio);
257         System.out.printf("MSE            : %.4f%n", mse);
258         System.out.printf("PSNR           : %.2f dB%n", psnr);
259         System.out.printf("Reconstructed   : %s%n", outPath);
260
261     } catch (IOException e) {
262         System.err.println("Error: " + e.getMessage());
263     }
264 }
265
266 // =====
267 // Batch analysis
268 // =====
269
270 static void runBatch(String imgPath) {
271     int[] levels = { 8, 16, 32 };
272     int[] types = { ORDER_1, ORDER_2, ADAPTIVE };
273     String[] names = { "Order-1", "Order-2", "Adaptive" };
274
275     try {
276         int[][] orig = readGrayscale(imgPath);
277         int h = orig.length, w = orig[0].length;
278
279         System.out.printf("%nBatch analysis on: %s (%dx%d)%n", imgPath, w, h);
280         System.out.printf("%-12s %-8s %10s %10s %12s %12s%n",
281             "Predictor", "Levels", "MSE", "PSNR(dB)", "Ratio",
282             "Orig(B)", "Enc(B)");
283         System.out.println("-".repeat(78));
284
285         for (int t = 0; t < types.length; t++) {
286             for (int L : levels) {
287                 int[][] rec = new int[h][w];
288                 int[][] q = encode(orig, types[t], L, rec);
289                 int[][] dec = decode(q, types[t], L);
290                 double mse = mse(orig, dec);

```

```

291         double psnr = psnr(mse);
292         double ratio = compressionRatio(L);
293         long ob = (long) h * w;
294         long eb = ((long) h * w * bitsPerIndex(L) + 7) / 8;
295         System.out.printf("%-12s %-8d %10.4f %10.2f %10.4fx %12d %12d%n",
296             names[t], L, mse, psnr, ratio, ob, eb);
297     }
298 }
299
300 } catch (IOException e) {
301     System.err.println("Error: " + e.getMessage());
302 }
303 }
304
305 // =====
306 // Main / CLI
307 // -----
308
309 public static void main(String[] args) {
310     Scanner sc = new Scanner(System.in);
311     while (true) {
312         System.out.println("\n+-----+");
313         System.out.println("| 2-D Feed Backward DPCM |");
314         System.out.println("| DSAI 325 - Assignment 5 | Spring 2026 |");
315         System.out.println("+-----+");
316         System.out.println("| 1. Encode & Decode Image |");
317         System.out.println("| 2. Batch Analysis (all predictors/levels)|");
318         System.out.println("| 3. Exit |");
319         System.out.println("+-----+");
320         System.out.print("Choice: ");
321         String ch = sc.nextLine().trim();
322
323         if (ch.equals("1")) {
324             System.out.print("Input image path: ");
325             String path = sc.nextLine().trim();
326
327             System.out.print("Predictor type (1=Order-1 2=Order-2 3=Adaptive): ");
328
329             int type = Integer.parseInt(sc.nextLine().trim());
330
331             System.out.print("Quantization levels (e.g. 8 16 32): ");
332             int L = Integer.parseInt(sc.nextLine().trim());
333
334             System.out.print("Output reconstructed image path: ");
335             String out = sc.nextLine().trim();
336
337             runSingle(path, type, L, out);
338
339         } else if (ch.equals("2")) {
340             System.out.print("Input image path: ");
341             String path = sc.nextLine().trim();
342             runBatch(path);
343
344         } else if (ch.equals("3")) {
345             System.out.println("Goodbye.");
346             break;
347         } else {
348             System.out.println("Invalid choice. Enter 1, 2, or 3.");
349         }
350     }
351     sc.close();
352 }

```

Listing 6: DPCM.java